JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Interface ConcurrentNavigableMap<K,V>

Type Parameters:

K - the type of keys maintained by this map

V - the type of mapped values

All Superinterfaces:

ConcurrentMap<K,V>, Map<K,V>, NavigableMap<K,V>, SequencedMap<K,V>, SortedMap<K,V>

All Known Implementing Classes:

ConcurrentSkipListMap

```
public interface ConcurrentNavigableMap<K,V>
extends ConcurrentMap<K,V>, NavigableMap<K,V>
```

A ConcurrentMap supporting NavigableMap operations, and recursively so for its navigable sub-maps.

This interface is a member of the Java Collections Framework.

Since:

1.6

Nested Class Summary

Nested classes/interfaces declared in interface Map
Map.Entry<K,V>

Method Summary

All Methods	Instance Methods	
Modifier and Type	**Method**	**Description**
NavigableSet<K>	**descendingKeySet**()	Returns a reverse order NavigableSet view of the keys contained in this map.
ConcurrentNavigableMap<K,V>	**descendingMap**()	Returns a reverse order view of the mappings contained in this map.
	headMap(K toKey)	Returns a view of the portion of this map

ConcurrentNavigableMap<K,V>		whose keys are strictly less than `toKey`.
ConcurrentNavigableMap<K,V>	**headMap**(K toKey, boolean inclusive)	Returns a view of the portion of this map whose keys are less than (or equal to, if `inclusive` is true) `toKey`.
NavigableSet<K>	**keySet**()	Returns a `NavigableSet` view of the keys contained in this map.
NavigableSet<K>	**navigableKeySet**()	Returns a `NavigableSet` view of the keys contained in this map.
ConcurrentNavigableMap<K,V>	**subMap**(K fromKey, boolean fromInclusive, K toKey, boolean toInclusive)	Returns a view of the portion of this map whose keys range from `fromKey` to `toKey`.
ConcurrentNavigableMap<K,V>	**subMap**(K fromKey, K toKey)	Returns a view of the portion of this map whose keys range from `fromKey`, inclusive, to `toKey`, exclusive.
ConcurrentNavigableMap<K,V>	**tailMap**(K fromKey)	Returns a view of the portion of this map whose keys are greater than or equal to `fromKey`.
ConcurrentNavigableMap<K,V>	**tailMap**(K fromKey, boolean inclusive)	Returns a view of the portion of this map whose keys are greater than (or equal to, if `inclusive` is true) `fromKey`.

Methods declared in interface ConcurrentMap

compute, computeIfAbsent, computeIfPresent, forEach, getOrDefault, merge, putIfAbsent, remove, replace, replace, replaceAll

Methods declared in interface Map

clear, containsKey, containsValue, equals, get, hashCode, isEmpty, put, putAll, remove, size

Methods declared in interface NavigableMap

ceilingEntry, ceilingKey, firstEntry, floorEntry, floorKey, higherEntry, higherKey, lastEntry, lowerEntry, lowerKey, pollFirstEntry, pollLastEntry, reversed

Methods declared in interface SequencedMap

sequencedEntrySet, sequencedKeySet, sequencedValues

Methods declared in interface SortedMap

comparator, entrySet, firstKey, lastKey, putFirst, putLast, values

Method Details

subMap

```
ConcurrentNavigableMap<K,V> subMap(K fromKey,
                                   boolean fromInclusive,
                                   K toKey,
                                   boolean toInclusive)
```

Description copied from interface: NavigableMap

Returns a view of the portion of this map whose keys range from `fromKey` to `toKey`. If `fromKey` and `toKey` are equal, the returned map is empty unless `fromInclusive` and `toInclusive` are both true. The returned map is backed by this map, so changes in the returned map are reflected in this map, and vice-versa. The returned map supports all optional map operations that this map supports.

The returned map will throw an `IllegalArgumentException` on an attempt to insert a key outside of its range, or to construct a submap either of whose endpoints lie outside its range.

Specified by:

`subMap` in interface `NavigableMap<K,V>`

Parameters:

`fromKey` - low endpoint of the keys in the returned map

`fromInclusive` - `true` if the low endpoint is to be included in the returned view

`toKey` - high endpoint of the keys in the returned map

`toInclusive` - `true` if the high endpoint is to be included in the returned view

Returns:

a view of the portion of this map whose keys range from `fromKey` to `toKey`

Throws:

`ClassCastException` - if `fromKey` and `toKey` cannot be compared to one another using this map's comparator (or, if the map has no comparator, using natural ordering). Implementations may, but are not required to, throw this exception if `fromKey` or `toKey` cannot be compared to keys currently in the map.

`NullPointerException` - if `fromKey` or `toKey` is null and this map does not permit null keys

`IllegalArgumentException` - if `fromKey` is greater than `toKey`; or if this map itself has a restricted range, and `fromKey` or `toKey` lies outside the bounds of the range

headMap

```
ConcurrentNavigableMap<K,V> headMap(K toKey,
                                    boolean inclusive)
```

Description copied from interface: `NavigableMap`

Returns a view of the portion of this map whose keys are less than (or equal to, if `inclusive` is true) `toKey`. The returned map is backed by this map, so changes in the returned map are reflected in this map, and vice-versa. The returned map supports all optional map operations that this map supports.

The returned map will throw an `IllegalArgumentException` on an attempt to insert a key outside its range.

Specified by:

`headMap` in interface `NavigableMap<K,V>`

Parameters:

`toKey` - high endpoint of the keys in the returned map

`inclusive` - `true` if the high endpoint is to be included in the returned view

Returns:

a view of the portion of this map whose keys are less than (or equal to, if `inclusive` is true) `toKey`

Throws:

`ClassCastException` - if `toKey` is not compatible with this map's comparator (or, if the map has no comparator, if `toKey` does not implement `Comparable`). Implementations may, but are not required to, throw this exception if `toKey` cannot be compared to keys currently in the map.

`NullPointerException` - if `toKey` is null and this map does not permit null keys

`IllegalArgumentException` - if this map itself has a restricted range, and `toKey` lies outside the bounds of the range

tailMap

```
ConcurrentNavigableMap<K,V> tailMap(K fromKey,
                                    boolean inclusive)
```

Description copied from interface: `NavigableMap`

Returns a view of the portion of this map whose keys are greater than (or equal to, if `inclusive` is true) `fromKey`. The returned map is backed by this map, so changes in the returned map are reflected in this map, and vice-versa. The returned map supports all optional map operations that this map supports.

The returned map will throw an `IllegalArgumentException` on an attempt to insert a key outside its range.

Specified by:

`tailMap` in interface `NavigableMap<K,V>`

Parameters:

`fromKey` - low endpoint of the keys in the returned map

`inclusive` - `true` if the low endpoint is to be included in the returned view

Returns:

a
view of the portion of this map whose keys are greater than (or equal to, if `inclusive` is true) `fromKey`

Throws:

`ClassCastException` - if `fromKey` is not compatible with this map's comparator (or, if the map has no comparator, if `fromKey` does not implement `Comparable`). Implementations may, but are not required to,

throw this exception if `fromKey` cannot be compared to keys currently in the map.

`NullPointerException` - if `fromKey` is null and this map does not permit null keys

`IllegalArgumentException` - if this map itself has a restricted range, and `fromKey` lies outside the bounds of the range

subMap

```
ConcurrentNavigableMap<K,V> subMap(K fromKey,
                                   K toKey)
```

Description copied from interface: `NavigableMap`

Returns a view of the portion of this map whose keys range from `fromKey`, inclusive, to `toKey`, exclusive. (If `fromKey` and `toKey` are equal, the returned map is empty.) The returned map is backed by this map, so changes in the returned map are reflected in this map, and vice-versa. The returned map supports all optional map operations that this map supports.

The returned map will throw an `IllegalArgumentException` on an attempt to insert a key outside its range.

Equivalent to `subMap(fromKey, true, toKey, false)`.

Specified by:

`subMap` in interface `NavigableMap<K,V>`

Specified by:

`subMap` in interface `SortedMap<K,V>`

Parameters:

`fromKey` - low endpoint (inclusive) of the keys in the returned map

`toKey` - high endpoint (exclusive) of the keys in the returned map

Returns:

a view of the portion of this map whose keys range from `fromKey`, inclusive, to `toKey`, exclusive

Throws:

`ClassCastException` - if `fromKey` and `toKey` cannot be compared to one another using this map's comparator (or, if the map has no comparator, using natural ordering). Implementations may, but are not required to, throw this exception if `fromKey` or `toKey` cannot be compared to keys currently in the map.

`NullPointerException` - if `fromKey` or `toKey` is null and this map does not permit null keys

IllegalArgumentException - if fromKey is greater than toKey; or if this map itself has a restricted range, and fromKey or toKey lies outside the bounds of the range

headMap

ConcurrentNavigableMap<K,V> headMap(K toKey)

Description copied from interface: NavigableMap

Returns a view of the portion of this map whose keys are strictly less than toKey. The returned map is backed by this map, so changes in the returned map are reflected in this map, and vice-versa. The returned map supports all optional map operations that this map supports.

The returned map will throw an IllegalArgumentException on an attempt to insert a key outside its range.

Equivalent to headMap(toKey, false).

Specified by:

headMap in interface NavigableMap<K,V>

Specified by:

headMap in interface SortedMap<K,V>

Parameters:

toKey - high endpoint (exclusive) of the keys in the returned map

Returns:

a view of the portion of this map whose keys are strictly less than toKey

Throws:

ClassCastException - if toKey is not compatible with this map's comparator (or, if the map has no comparator, if toKey does not implement Comparable). Implementations may, but are not required to, throw this exception if toKey cannot be compared to keys currently in the map.

NullPointerException - if toKey is null and this map does not permit null keys

IllegalArgumentException - if this map itself has a restricted range, and toKey lies outside the bounds of the range

tailMap

ConcurrentNavigableMap<K,V> tailMap(K fromKey)

Description copied from interface: NavigableMap

Returns a view of the portion of this map whose keys are greater than or equal to `fromKey`. The returned map is backed by this map, so changes in the returned map are reflected in this map, and vice-versa. The returned map supports all optional map operations that this map supports.

The returned map will throw an `IllegalArgumentException` on an attempt to insert a key outside its range.

Equivalent to `tailMap(fromKey, true)`.

Specified by:

`tailMap` in interface `NavigableMap<K,V>`

Specified by:

`tailMap` in interface `SortedMap<K,V>`

Parameters:

`fromKey` - low endpoint (inclusive) of the keys in the returned map

Returns:

a view of the portion of this map whose keys are greater than or equal to `fromKey`

Throws:

`ClassCastException` - if `fromKey` is not compatible with this map's comparator (or, if the map has no comparator, if `fromKey` does not implement `Comparable`). Implementations may, but are not required to, throw this exception if `fromKey` cannot be compared to keys currently in the map.

`NullPointerException` - if `fromKey` is null and this map does not permit null keys

`IllegalArgumentException` - if this map itself has a restricted range, and `fromKey` lies outside the bounds of the range

descendingMap

`ConcurrentNavigableMap<K,V> descendingMap()`

Returns a reverse order view of the mappings contained in this map. The descending map is backed by this map, so changes to the map are reflected in the descending map, and vice-versa.

The returned map has an ordering equivalent to `Collections.reverseOrder(comparator())`. The expression `m.descendingMap().descendingMap()` returns a view of `m` essentially equivalent to `m`.

Specified by:

descendingMap in interface NavigableMap<K,V>

Returns:

a reverse order view of this map

navigableKeySet

NavigableSet<K> navigableKeySet()

Returns a NavigableSet view of the keys contained in this map. The set's iterator returns the keys in ascending order. The set is backed by the map, so changes to the map are reflected in the set, and vice-versa. The set supports element removal, which removes the corresponding mapping from the map, via the `Iterator.remove`, `Set.remove`, `removeAll`, `retainAll`, and `clear` operations. It does not support the `add` or `addAll` operations.

The view's iterators and spliterators are *weakly consistent*.

Specified by:

navigableKeySet in interface NavigableMap<K,V>

Returns:

a navigable set view of the keys in this map

keySet

NavigableSet<K> keySet()

Returns a NavigableSet view of the keys contained in this map. The set's iterator returns the keys in ascending order. The set is backed by the map, so changes to the map are reflected in the set, and vice-versa. The set supports element removal, which removes the corresponding mapping from the map, via the `Iterator.remove`, `Set.remove`, `removeAll`, `retainAll`, and `clear` operations. It does not support the `add` or `addAll` operations.

The view's iterators and spliterators are *weakly consistent*.

This method is equivalent to method `navigableKeySet`.

Specified by:

keySet in interface Map<K,V>

Specified by:

keySet in interface SortedMap<K,V>

Returns:

a navigable set view of the keys in this map

descendingKeySet

`NavigableSet<K> descendingKeySet()`

Returns a reverse order `NavigableSet` view of the keys contained in this map. The set's iterator returns the keys in descending order. The set is backed by the map, so changes to the map are reflected in the set, and vice-versa. The set supports element removal, which removes the corresponding mapping from the map, via the `Iterator.remove`, `Set.remove`, `removeAll`, `retainAll`, and `clear` operations. It does not support the `add` or `addAll` operations.

The view's iterators and spliterators are *weakly consistent*.

Specified by:

`descendingKeySet` in interface `NavigableMap<K,V>`

Returns:

a reverse order navigable set view of the keys in this map

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.